

Arrays and tables: NumPy and pandas

One block of numbers with a shape, then labelled rows and columns on top of it.

THE QUESTION

How do we compute on thousands of numbers, or on a table with named columns, without a Python loop for every step?

THE INTUITION

A NumPy array is one block of numbers of one type, with a shape. An operation on the array runs on every element at once. A pandas DataFrame adds labels to the rows and columns, so you select, group and join by name.

KEY TAKEAWAY

Know the shape and name the axis. Then let vectorised operations, masks and groupby do the looping for you.

list → array: one dtype → shape, axis → vectorised ops → mask → DataFrame → groupby, merge

1 An array holds one type in one block

list [1, 2, 3] * 2 → [1, 2, 3, 1, 2, 3] repeats the list
array np.array([1, 2, 3]) * 2 → [2, 4, 6] doubles each value

np.array([1, 2, 3.5])
ints upcast: one dtype

1.0	2.0	3.5
-----	-----	-----

dtype float64
nbytes = 24
8 bytes 8 bytes 8 bytes

square 1,000,000 numbers measured once; varies by machine

[x*x for x in lst] 17 ms
arr * arr 0.6 ms

np.array(lst) copies the values into one block, so every element shares one dtype (int64, float64, bool); ints mixed with floats become float64. Arithmetic runs element by element in compiled code, so arr * arr beats the list comprehension. A list holds any Python objects; its * and + repeat or join.

2 The shape counts elements along each axis

(3,) 1-D vector
(1, 3) 1 row, 3 cols
(3, 1) 3 rows, 1 col

np.zeros(3) ndim 1
np.zeros((1, 3)) ndim 2
np.zeros((3, 1)) ndim 2

np.arange(6) → .reshape(2, -1) → (2, 3)

-1: work it out; fills row by row

Check a.shape, a.ndim, a.size and a.dtype first. (3,), (1, 3) and (3, 1) hold the same numbers but broadcast differently (panel 5). reshape changes the shape, never the data; the sizes must multiply to the same total. a.reshape(-1) flattens. Builders: np.zeros, np.ones, np.arange(0, 10, 2), np.linspace(0, 1, 5), np.eye(3).

3 Index with [row, col]; a slice is a view

	0	1	2	3
0	7	9	4	8
1	3	6	8	7
2	8	9	9	6

stop is excluded: :2 means 0 and 1

M[0, 2] → 4
row 0, col 2

M[1:, :2] → (2, 2) block
rows 1 on, cols 0 to 1

M[:, -1] → [8, 7, 6]
last column

M holds 3 students' marks on 4 quizzes (illustrative). Slices use start:stop:step; negative indices count from the end. A slice is a view: after sub = M[1:, :2]; sub[0, 0] = 0, M has changed too. Use .copy() for an independent array. np.vstack stacks rows, np.hstack side by side.

4 axis=0 runs down the rows

M, shape (3, 4)

s1	7	9	4	8
s2	3	6	8	7
s3	8	9	9	6

axis 0 ↓

7	9	4	8
3	6	8	7
8	9	9	6

M.mean(axis=1) → shape (3,)
one mean per student (row)

M.mean(axis=0) → shape (4,)
all 12 values

Operators and ufuncs such as np.sqrt act element by element. Aggregations (sum, mean, std, max, argmax) collapse the axis you name: axis=0 leaves one value per column, axis=1 one per row. np.argmax(M[:, 0]) is 2, the position of the top mark on quiz 0. np.std divides by n, pandas by n - 1.

5 Broadcasting stretches axes of size 1

M (3, 4) mean (4,) (3, 4)

7	9	4	8	6	8	7	7	1	1	-3	1
3	6	8	7	6	8	7	7	-3	-2	1	0
8	9	9	6	6	8	7	7	2	1	2	-1

dashed rows: stretched, not copied

each column now has mean 0

Line the shapes up on the right; each pair must be equal or contain a 1.

(3, 4) with (4) → (3, 4) M - M.mean(axis=0)
(3, 4) with (3, 1) → (3, 4) M - M.mean(axis=1, keepdims=True)
(3, 4) with (3) error M - M.mean(axis=1): 4 vs 3

Subtracting the column means centres every quiz with no loop: the (4,) means act as a (1, 4) row repeated for each student. Shapes are compared from the last axis back; a missing axis counts as 1. Per-row values need shape (3, 1): use keepdims=True or v[:, None].

6 A boolean mask selects by condition

M M > 7 (bool)

7	9	4	8	F	T	F	T
3	6	8	7	F	F	T	F
8	9	9	6	T	T	T	F

M[M > 7] → [9, 8, 8, 8, 9, 9]

(M > 7).sum() → 6

(M > 7).mean() → 0.50

M > 7 is a True/False array of the same shape; indexing with it keeps the matching values as a 1-D array. True counts as 1, so .sum() counts and .mean() gives the fraction: 6 of 12 marks. Combine conditions with &, | and ~, with brackets round each one: M[(M > 5) & (M < 9)]. np.where(M > 7, M, 0) keeps the shape.

Tables with labels: pandas

NumPy columns that share one index. Six made-up students (illustrative) run through every example.

7 A DataFrame is columns that share an index

	name	section	quiz	lab
index	Asha	A	6	8
	Bilal	B	9	7
	Chen	A	4	6
	Divya	B	7	10
	Emil	A	8	9
	Farah	B	8	NaN
dtype	object	object	int64	float64

`df["quiz"]`
is a Series:
values + the same
index

`df.shape` → (6, 3)
lab is float64:
NaN is a float

Build one from a dict of lists or `pd.read_csv(path)`; `set_index("name")` makes names the row labels. Each column is a Series with its own dtype (text is object). Look first with `df.head()`, `df.info()`, `df.dtypes`.

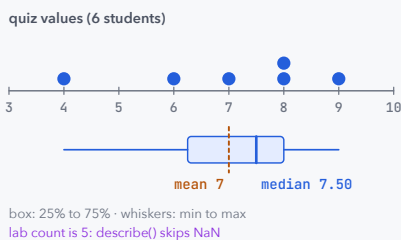
8 loc uses labels, iloc uses positions

<code>loc["Chen", "quiz"]</code>				<code>iloc[0:2]</code>				<code>df[df.quiz > 7]</code>			
	sec	quiz	lab		sec	quiz	lab		sec	quiz	lab
Asha	A	6	8	Asha	A	6	8	Asha	A	6	8
Bilal	B	9	7	Bilal	B	9	7	Bilal	B	9	7
Chen	A	4	6	Chen	A	4	6	Chen	A	4	6
Divya	B	7	10	Divya	B	7	10	Divya	B	7	10
Emil	A	8	9	Emil	A	8	9	Emil	A	8	9
Farah	B	8	NaN	Farah	B	8	NaN	Farah	B	8	NaN
labels (loc)				positions; 2 excluded				rows where the mask is True			

`df.loc["Chen", "quiz"]` is 4. `iloc[0:2]` stops before position 2, but a `loc` slice keeps its end: `df.loc["Asha":"Chen"]` has 3 rows. `df.query("quiz > 7")` equals the mask filter, and ipywidgets' interact can drive it with a slider.

9 describe() summarises each column

	quiz	lab
count	6	5
mean	7	8
std	1.79	1.58
min	4	6
25%	6.25	7
50%	7.50	8
75%	8	9
max	9	10



The quiz mean (7) sits below the median (7.5) because one low mark, 4, pulls it down. Quartiles interpolate, hence 6.25. std divides by $n - 1$ (1.79; `np.std` gives 1.63). For text columns use `value_counts()`.

13 Pause and think

Dtype of `np.array([1, 2, "3"])`? Text (<U21) for all three, so `+ 1` fails. `a.shape` is (5,). What is `a.reshape(-1, 1).shape`? (5, 1), a column.

Do (4, 1) and (3,) broadcast? Yes, to (4, 3): each aligned pair has a 1. `df.loc[0:2]` or `df.iloc[0:2]` on the default index? `loc` includes label 2 (3 rows); `iloc` gives 2 rows.

Why is `M[M > 7]` 1-D? The kept values need not form a rectangle. An int column became float after a left merge. Why? Some keys found no match and got NaN, a float.

10 groupby splits, applies, then combines

rows	split by section	
Asha A 6	section A quiz: 6, 4, 8 lab: 8, 6, 9	quiz mean 6 lab mean 7.67 (lab count 3)
Bilal B 9		
Chen A 4		
Divya B 7	section B quiz: 9, 7, 8 lab: 7, 10, NaN	quiz mean 8 lab mean 8.50 (lab count 2)
Emil A 8		
Farah B 8		
section · quiz		apply mean to each group, combine into one table

`df.groupby("section")["quiz"].mean()` gives 6 for A and 8 for B: a loop over sections with a mask, in one line. `.agg(["count", "mean"])` gives several numbers per group; `pivot_table` lays two keys out as a grid.

11 merge joins two tables on a key

marks (left)			projects			merge(on="name", how="left")		
name	quiz		name	project		name	quiz	project
Asha	6		Asha	15		Asha	6	15
Bilal	9		Bilal	18		Bilal	9	18
Chen	4		Chen	12		Chen	4	12
Divya	7		Divya	16		Divya	7	16
Emil	8		Emil	14		Emil	8	14
Farah	8		Gita	17		Farah	8	NaN

rows: left 6 · inner 5 (matches only) · outer 7 (all names)
Gita has a project but no marks; Farah has marks but no project.

`pd.merge(marks, projects, on="name", how="left")` matches rows by name, like a database join. Count rows after every merge to see which keys failed to match. `pd.concat([a, b], ignore_index=True)` stacks tables instead.

12 Missing values change the answer

<code>df["lab"]</code>	call	lab mean (scale 0 to 10)
Asha 8	<code>mean()</code>	8
Bilal 7	skips NaN, 5 values	
Chen 6	<code>fillna(0).mean()</code>	6.67
Divya 10	a 0 drags it down	
Emil 9	<code>fillna(mean).mean()</code>	8
Farah NaN	same mean, less spread	
<code>isna().sum() → 1</code>	<code>dropna()</code> : drops Farah's row → 5 rows	

A gap is stored as NaN, which makes an int column float. `df.isna().sum()` counts gaps; `mean()` and `describe()` skip them. Filling with 0 claims Farah scored 0 and drops the mean from 8 to 6.67; filling with the mean keeps 8 but shrinks the spread. Pick the rule on purpose and report it.

check shape and dtype first · name the axis · masks instead of if-loops · count NaNs and rows after a merge

Source: the course notebooks on NumPy, Pandas and widgets, nipunbatra.github.io/psdv-teaching, used for the list of topics. All examples and numbers are our own (illustrative) and computed by the sheet's figure script; the timing in panel 1 is one measurement and varies by machine.